# CODEATLAS

mini-project of RDBMS

## PROJECT REPORT

By Aditi Ranjan (2502051464)

Course : Relational Database Management Systems (RDBMS)
B.Tech – Computer Science and Engineering
Manipal University Jaipur

*This report explains the project in simple language.*

# Contents

1. Introduction
2. What is CodeAtlas?
3. Why This Project Was Made
4. Main Technologies Used
5. How the Project Works
6. Database Design
7. Tables Used in the Project
8. SQL Concepts Used
9. Views, Indexes, Procedures and Triggers
10. Reports Available in the Project
11. C++ Application and Menu
12. Project Folder Structure
13. How to Set Up and Run the Project
14. Testing and Important Notes
15. Current Limitations
16. Conclusion

- ER diagram and table structure

# 1. Introduction

Competitive programming is a way of solving coding problems on websites such as codeforces, atcoder and leetcode. People use these platforms to practise programming, take part in contests and improve their problem-solving skills.

When a person solves many problems, a lot of information is created. This can include the problem name, its difficulty, the programming language used, the result of a submission, the time taken, and the contests in which the person took part.

CodeAtlas is a database project that stores sample information of this kind and uses SQL queries to show useful summaries. The project also has a C++ application that connects to the MySQL database and displays selected information.

# 2. What is CodeAtlas?

CodeAtlas is a Competitive Programming Database and Analytics project. In simple words, it is a system that keeps online coding platform related information in an organised form and helps us ask questions about that information.

For example, it can show how many problems are listed for each platform, how many submissions were accepted, which problems were attempted the most, and which programming languages were used in the sample data.

The project uses MySQL to store the data, SQL to work with the data, and C++ to provide a small menu-based application.

# 3. Why This Project Was Made

The main purpose of codeatlas is to practise database concepts through a topic related to programming.

- To store competitive programming information in properly connected tables.
- To understand how tables are linked using *primary keys* and *foreign keys*.
- To practise inserting, updating, deleting and retrieving records.
- To use SQL joins, subqueries and aggregate functions to get useful results.
- To understand database features such as views, indexes, transactions, procedures and triggers.
- To connect a c++ program with a MySQL database and display information.

# 4. Main Technologies Used

| Technology | How it is used |
|---|---|
| **C++** | Used to create the menu-based application. |
| **MySQL** | Stores the project data in tables. |
| **SQL** | Used to create tables and work with the stored data. |
| **MySQL Connector/C++** | Allows the C++ program to communicate with MySQL. |
| **CMake** | Helps configure and build the C++ program. |
| **OpenSSL** | A supporting library used by the current Connector/C++ setup. |
| **Git** | Can be used to track project changes and upload the project to GitHub. |

# 5. How the Project Works

The project has two main parts:

### A. MySQL Database

The database stores the information. It contains tables for users, platforms, problems, contests, submissions, programming languages and other related details. The tables are connected so that related information can be found together.

### B. C++ Application

The C++ program connects to the MySQL database. It shows a menu to the user. When the user selects an option, the program runs the required database queries and displays the result in the terminal.

In short: MySQL stores the data, SQL works with the data, and C++ provides a way to view selected information.

# 6. Database Design

Database design means deciding what information should be stored and how it should be organised. Instead of putting everything in one large table, CodeAtlas divides information into smaller tables.

For example, platform details are stored separately from problem details. A problem can then be connected to a platform using an ID. This makes the database easier to manage and reduces repeated information.

A primary key is a column that uniquely identifies each row in a table. A foreign key is a column that connects one table to another table. These keys help keep the data organised and connected.

# 7. Tables Used in the Project

The database design contains 12 main tables. The trigger section also creates an additional table named 'submission_audit' to keep a record when a new submission is added.

| Table name | What it stores |
|---|---|
| **users** | Details about users in the sample system. |
| **platforms** | Names of competitive programming platforms. |
| **platform_accounts** | The accounts users have on different platforms. |
| **problems** | Problem names and related problem details. |
| **tag** | Problem topics or categories. |
| **problem_tag** | Connects problems with their tags. |
| **contests** | Contest details. |
| **contest_problem** | Connects contests with the problems included in them. |
| **contest_participation** | Stores which users participated in contests. |
| **languages** | Programming languages used for submissions. |
| **verdicts** | Possible results of a submission, such as accepted or wrong answer. |
| **submissions** | Details about users' attempts to solve problems. |
| **submission_audit** | Stores an audit entry when a new submission is inserted by the trigger. |

# 8. SQL Concepts Used

The project includes SQL examples for the main topics normally covered in a relational database course.

| SQL concept | Simple meaning |
|---|---|
| **CREATE** | Creates the database tables and other database objects. |
| **INSERT** | Adds new records to a table. |
| **SELECT** | Reads and displays information from tables. |
| **UPDATE** | Changes information that is already stored. |
| **DELETE** | Removes selected records. |
| **ALTER** | Changes the structure of an existing table. |
| **TRUNCATE** | Removes all rows from a table while keeping the table structure. |
| **Constraints** | Rules such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL and CHECK help protect data quality. |
| **Joins** | Combines related information from two or more tables. |
| **Subqueries** | Uses the result of one query inside another query. |
| **Aggregate functions** | Functions such as COUNT, SUM, AVG, MIN and MAX calculate summaries. |
| **GROUP BY and HAVING** | Groups rows and filters groups based on a condition. |
| **Set operations** | Combine results from compatible SELECT queries where supported by the database. |
| **Transactions** | COMMIT saves a transaction, ROLLBACK cancels changes, and SAVEPOINT marks a point to return to. |
| **Privileges** | GRANT and REVOKE are used to give or remove database permissions. |

# 9. Views, Indexes, Procedures and Triggers

## Views

A view is like a saved query. Instead of writing the same SELECT query again and again, we can save it as a view and read from it when needed. The project creates these views:

- problem_platform_view
- user_submission_view
- accepted_submissions_view

## Indexes

An index helps MySQL find records faster in some searches. It is similar to an index at the back of a book. The project includes indexes named:

- idx_problem_name
- idx_submission_user
- idx_submission_user_date

## Stored Procedures

A stored procedure is a saved group of SQL statements that can be run when needed. The project includes:

- get_all_users()
- get_problems_by_difficulty(p_difficulty)

- get_user_submissions(p_user_id)

### Triggers

A trigger is a database action that runs automatically when a particular event happens. The project includes:

- before_submission_insert: checks a submission before it is inserted and rejects a negative execution time.
- after_submission_insert: adds an entry to submission_audit after a new submission is inserted.
- before_rating_update: prevents a rating from being changed to a negative value.

## 10. Reports Available in the Project

The SQL report file contains 10 SELECT queries. These reports read the sample data and show it in a more useful form.

| Report | What it tells us |
|---|---|
| **Platform-wise Problem Counts** | Shows how many problems are present for each platform. |
| **Submission Verdict Distribution** | Shows the number of submissions for each result or verdict. |
| **User Submission Activity** | Shows submission activity for users. |
| **Most Attempted Problems** | Shows which problems have the highest number of attempts. |
| **Platform Acceptance Rates** | Shows the acceptance rate for each platform based on the stored sample records. |
| **Programming Language Usage** | Shows how often each programming language appears in submissions. |
| **Contest Participation Summary** | Shows a summary of user participation in contests. |
| **Problems by Difficulty** | Groups problems according to their difficulty. |
| **Topic-wise Problem Counts** | Shows how many problems are connected to each topic or tag. |
| **Active Users** | Shows users who have activity in the stored submission data. |

## 11. C++ Application and Menu

The current C++ application has three main options:

1. Project Overview: displays summary information from the database, such as counts of users, platforms, problems, contests, submissions, tables and views.
2. Database Design and Modelling: displays information about the database tables, their columns, primary keys, foreign key relationships and views.
3. Run MySQL Report: reads the SQL report file and lets the user select and run one of the available reports.

The application also has an exit option. The report menu allows the user to return to the main menu or exit. The application is designed as a terminal-based program, so it does not currently have a graphical user interface.

# 12. Project Folder Structure

```
CodeAtlas/
│
├── core/
│   └── main.cpp
│
├── sql/
│   ├── 01_create_table.sql
│   ├── 02_constraints.sql
│   ├── 03_alter_operations.sql
│   ├── 04_seed_data.sql
│   ├── 05_dml_operations.sql
│   ├── 06_joins.sql
│   ├── 07_subqueries.sql
│   ├── 08_views.sql
│   ├── 09_indexes.sql
│   ├── 10_transactions.sql
│   ├── 11_privileges.sql
│   ├── 12_procedures.sql
│   ├── 13_triggers.sql
│   ├── 14_reports.sql
│   └── 15_final_verification.sql
│
├── build/
├── .vscode/
├── CMakeLists.txt
└── .gitignore
```

# 13. How to Set Up and Run the Project

## Required dependencies

- A C++ compiler such as clang or GCC.
- CMake version 3.20 or above.
- MySQL server.
- MySQL connector/C++.
- OpenSSL 3 for the current connector/C++ setup.
- Git, if you want to clone the project repository.

## macOS installation

On macOS, the dependencies used for the current setup can be installed with homebrew:

```
brew install cmake mysql mysql-connector-c++ openssl@3
```

Homebrew is mainly used on macOS, but it also supports Linux. Windows users can install the dependencies using suitable package managers or manual installers.

## Database setup

Create the CodeAtlas database in MySQL and execute the SQL files in the correct order. The numbered file names show the intended order, starting with table creation and constraints, followed by sample data and the remaining SQL features.

The seed data is demonstration data. The project does not automatically download live user, problem or contest data from Codeforces, AtCoder or LeetCode.

Some scripts create objects that may already exist if they have been run before. Before running the scripts again, check their contents and the current database state rather than blindly executing every file a second time.

### Build and run

Open Terminal in the project folder and run:

```
cmake -S . -B build

cmake --build build

./build/CodeAtlas
```

The current C++ connection uses MySQL X Protocol on port 33060. Make sure the MySQL server is running and that the X Plugin is enabled. The host, port, username and password in the connection code must match the local MySQL setup.

## 14. Testing and Important Notes

- The project includes a final verification SQL file to help check the database objects and stored data.
- The report menu runs SELECT statements from sql/14_reports.sql.
- The overview and database design options read information directly from MySQL.
- The sample records are used to demonstrate database operations and reports.
- The current CMake configuration was prepared for a macOS setup using Homebrew. Other systems may need configuration changes.

## 15. Current Limitations

- The project uses sample data and does not fetch live information from competitive programming websites.
- The application is terminal-based and does not currently have a graphical interface.
- The current CMake file contains machine-specific Homebrew paths.
- Some database features are demonstrated through SQL scripts rather than separate options in the C++ menu.
- The project focuses on relational database concepts and does not currently implement a NoSQL database.

## 16. Conclusion

CodeAtlas is a database project. It stores related information in separate MySQL tables and uses SQL queries to find useful results from that information. The project covers important database topics such as keys, constraints, joins, subqueries, views, indexes, transactions, privileges, stored procedures and triggers.

The C++ application connects to the database and gives the user a simple menu to view an overview, inspect the database design and run reports. Overall, the project is a practical way to understand how a relational database is designed, how SQL is used, and how a C++ program can work with a MySQL database.

# Project Summary

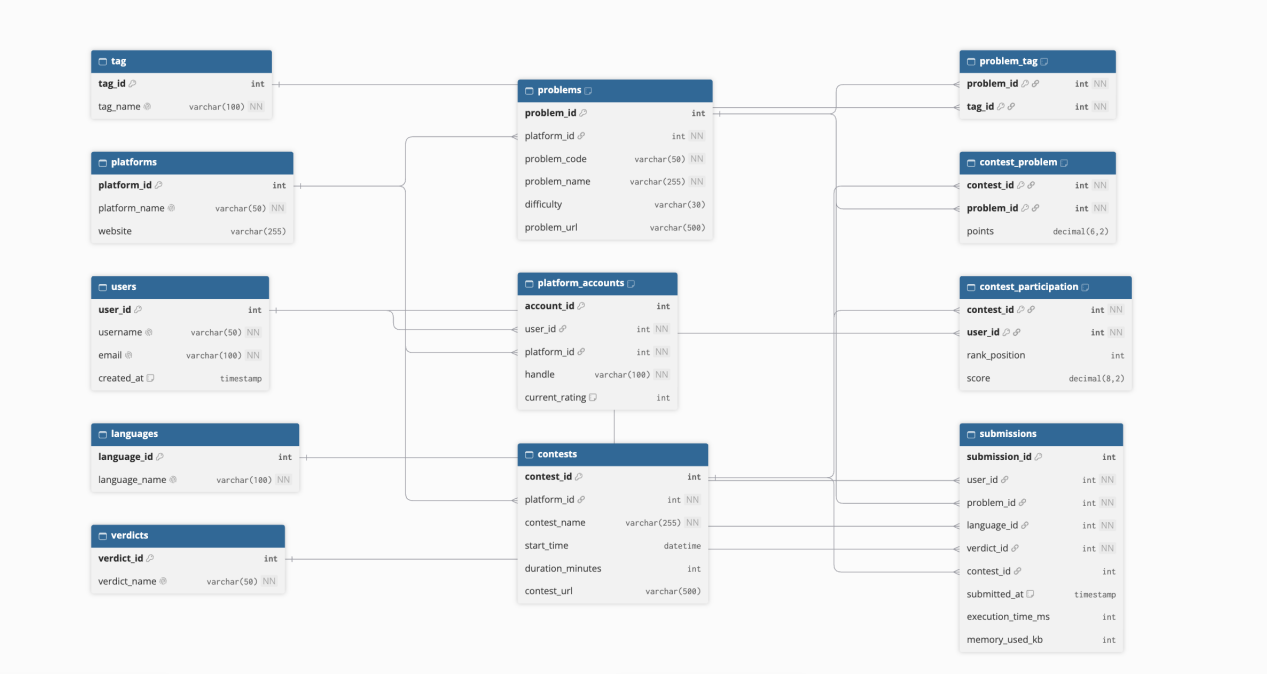
CodeAtlas is like a digital record book for competitive programming information. MySQL keeps the records, SQL helps organise and study them, and C++ gives us a menu to see the information. The project uses example data to show how a proper database can be created and how different database concepts work together.

# ER Diagram andTable Structure

This appendix contains the ER diagram image included in the uploaded CodeAtlas project and a simple description of the columns in each database table. The column names, data types and key information below are taken from sql/01_create_table.sql. The audit table is taken from sql/13_triggers.sql.

## 1. Entity-Relationship (ER) Diagram

The ER diagram is a picture of the database design. Each box represents a table, and the connecting lines show how tables are related. PK means Primary Key and FK means Foreign Key.




*ER diagram – made by using dbdigram.io*

## 2. Table Structure

A data dictionary explains what each column stores, its data type and whether it is a key or has a special rule. INT and BIGINT store whole numbers, VARCHAR stores text, DECIMAL stores numbers with decimal places, DATETIME stores date and time, and TIMESTAMP stores a timestamp.

### users

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **user_id** | INT | Primary key; automatically generated |
| **username** | VARCHAR(50) | Required; unique username |
| **email** | VARCHAR(100) | Required; unique email |
| **created_at** | TIMESTAMP | Defaults to the current timestamp |

### platforms

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **platform_id** | INT | Primary key; automatically generated |
| **platform_name** | VARCHAR(50) | Required; unique platform name |
| **website** | VARCHAR(200) | Required; unique website |

### platform_accounts

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **account_id** | INT | Primary key; automatically generated |
| **user_id** | INT | Required; foreign key to users |
| **platform_id** | INT | Required; foreign key to platforms |
| **handle** | VARCHAR(100) | Required; account handle on that platform |
| **current_rating** | INT | Defaults to 0; current platform rating |

### problems

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **problem_id** | INT | Primary key; automatically generated |
| **platform_id** | INT | Required; foreign key to platforms |
| **problem_code** | VARCHAR(50) | Required; problem code |
| **problem_name** | VARCHAR(200) | Required; problem name |
| **difficulty** | VARCHAR(20) | Optional; problem difficulty |
| **problem_url** | VARCHAR(200) | Optional; problem link |

### tag

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **tag_id** | INT | Primary key; automatically generated |
| **tag_name** | VARCHAR(50) | Required; unique topic name |

### problem_tag

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **problem_id** | INT | Part of composite primary key; foreign key to problems |
| **tag_id** | INT | Part of composite primary key; foreign key to tag |

### contests

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **contest_id** | INT | Primary key; automatically generated |
| **platform_id** | INT | Required; foreign key to platforms |
| **contest_name** | VARCHAR(150) | Required; contest name |
| **start_time** | DATETIME | Optional; contest start date and time |
| **duration_minutes** | INT | Optional; contest length in minutes |
| **contest_url** | VARCHAR(300) | Optional; contest link |

### contest_problem

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **contest_id** | INT | Part of composite primary key; foreign key to contests |
| **problem_id** | INT | Part of composite primary key; foreign key to problems |
| **points** | DECIMAL(6,2) | Optional; points for the problem |

### contest_participation

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **contest_id** | INT | Part of composite primary key; foreign key to contests |
| **user_id** | INT | Part of composite primary key; foreign key to users |
| **rank_position** | INT | Optional; user's rank in the contest |
| **score** | DECIMAL(8,2) | Optional; user's contest score |

### languages

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **language_id** | INT | Primary key; automatically generated |
| **language_name** | VARCHAR(100) | Required; unique language name |

### verdicts

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **verdict_id** | INT | Primary key; automatically generated |
| **verdict_name** | VARCHAR(100) | Required; unique submission result |

### submissions

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **submission_id** | BIGINT | Primary key; automatically generated |
| **user_id** | INT | Required; foreign key to users |
| **problem_id** | INT | Required; foreign key to problems |
| **language_id** | INT | Required; foreign key to languages |
| **verdict_id** | INT | Required; foreign key to verdicts |
| **contest_id** | INT | Optional; foreign key to contests; can be NULL |
| **submitted_at** | DATETIME | Defaults to the current timestamp |
| **execution_time_ms** | INT | Optional; execution time in milliseconds |
| **memory_used_kb** | INT | Optional; memory used in KB |

### submission_audit (created by trigger script)

| Column name | Data type | Key / rule / simple meaning |
|---|---|---|
| **audit_id** | INT | Primary key; automatically generated |
| **submission_id** | BIGINT | ID of the related submission |
| **action_type** | VARCHAR(20) | Action recorded, for example insert |
| **action_time** | TIMESTAMP | Defaults to the current timestamp |

Note: the schema file creates 12 main table, additional submission_audit table is created separately by the trigger script, so it is included here as well.

## 3. Main Relationships Between Tables

The following points explain the main table connections in simple words:

| Tables | Relationship explained simply |
|---|---|
| **users and platform_accounts** | One user can have multiple platform accounts. |
| **platforms and platform_accounts** | One platform can be linked to many user accounts. |
| **platforms and problems** | One platform can contain many problems. |
| **platforms and contests** | One platform can host many contests. |
| **problems and tag** | A problem can have many tags, and one tag can belong to many problems. The problem_tag table connects them. |
| **contests and problems** | A contest can contain many problems, and a problem can appear in different contests. The contest_problem table connects them. |
| **users and contests** | Users can participate in many contests. The contest_participation table stores this information. |
| **users, problems, languages and verdicts** | Each submission is linked to one user, one problem, one programming language and one verdict. |
| **contests and submissions** | A submission may be linked to a contest. The contest_id can be empty when the submission was made outside a contest. |

***Future Scope***: I would like build the database from scratch and implementing it in CodeAtlas. I want to understand and implement what actually happens behind the scenes, from storing data and managing tables to handling queries and relationships. Just to get a better understanding of how a database system works internally.